Name- Bhakare Purva Shivaji

Roll no- 08

Practical no- 06

```
# California Housing Dataset
# 1. Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
# 2. Read the dataset
file_path = r"C:\Users\PVGIT\Downloads\california_housing_test.csv"
housing = pd.read_csv(file_path)
print("Dataset loaded successfully!")
print("Shape of dataset:", housing.shape)
# Display first 5 rows
print("\nFirst 5 rows of dataset:")
print(housing.head())
# 3. Data Visualization - Histograms
housing.hist(figsize=(12, 10), bins=30)
plt.suptitle("Histograms of California Housing Dataset")
plt.tight_layout()
plt.show()
# 4. Data Visualization - Boxplots
housing.plot(
    kind='box',
    subplots=True,
    layout=(3, 3),
```

```
    figsize=(15, 10),
    sharex=False,
    sharey=False
)
plt.suptitle("Boxplots of California Housing Dataset")
plt.tight_layout()
plt.show()
# 5. Select input and output variables
# X = Median Income
# y = Median House Value
X = housing[['median_income']]
y = housing['median_house_value']
# 6. Split dataset into training and testing sets
# 80% Training and 20% Testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("\nTraining data size:", X_train.shape)
print("Testing data size:", X_test.shape)
# 7. LINEAR REGRESSION
linear_model = LinearRegression()
# Train the model
linear_model.fit(X_train, y_train)
# Predict values
y_pred_linear = linear_model.predict(X_test)
# Calculate performance
mse_linear = mean_squared_error(y_test, y_pred_linear)
mae_linear = mean_absolute_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)
```

```
print("\n========== LINEAR REGRESSION ==========")
print("MSE:", mse_linear)
print("MAE:", mae_linear)
print("R2 Score:", r2_linear)
# 8. POLYNOMIAL REGRESSION
# Degree 2 polynomial
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
poly_model = LinearRegression()
# Train the polynomial model
poly_model.fit(X_train_poly, y_train)
# Predict values
y_pred_poly = poly_model.predict(X_test_poly)
# Calculate performance
mse_poly = mean_squared_error(y_test, y_pred_poly)
mae_poly = mean_absolute_error(y_test, y_pred_poly)
r2_poly = r2_score(y_test, y_pred_poly)
print("\n========== POLYNOMIAL REGRESSION ==========")
print("MSE:", mse_poly)
print("MAE:", mae_poly)
print("R2 Score:", r2_poly)
# 9. COMPARISON OF BOTH MODEL
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Polynomial Regression'],
    'MSE': [mse_linear, mse_poly],
    'MAE': [mae_linear, mae_poly],
    'R2 Score': [r2_linear, r2_poly]
})
print("\n========== MODEL COMPARISON ==========")
print(results)
```

```
# 10. VISUALIZATION OF LINEAR REGRESSION
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, alpha=0.5, label='Actual')
plt.scatter(X_test, y_pred_linear, alpha=0.5, label='Predicted')
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Linear Regression")
plt.legend()
plt.show()
# 11. VISUALIZATION OF POLYNOMIAL REGRESSION
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, alpha=0.5, label='Actual')
plt.scatter(X_test, y_pred_poly, alpha=0.5, label='Predicted')
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Polynomial Regression")
plt.legend()
plt.show()
# 12. Display the better model
if r2_poly > r2_linear:
    print("\nPolynomial Regression performs better based on R2 Score.")
else:
    print("\nLinear Regression performs better based on R2 Score.")
```

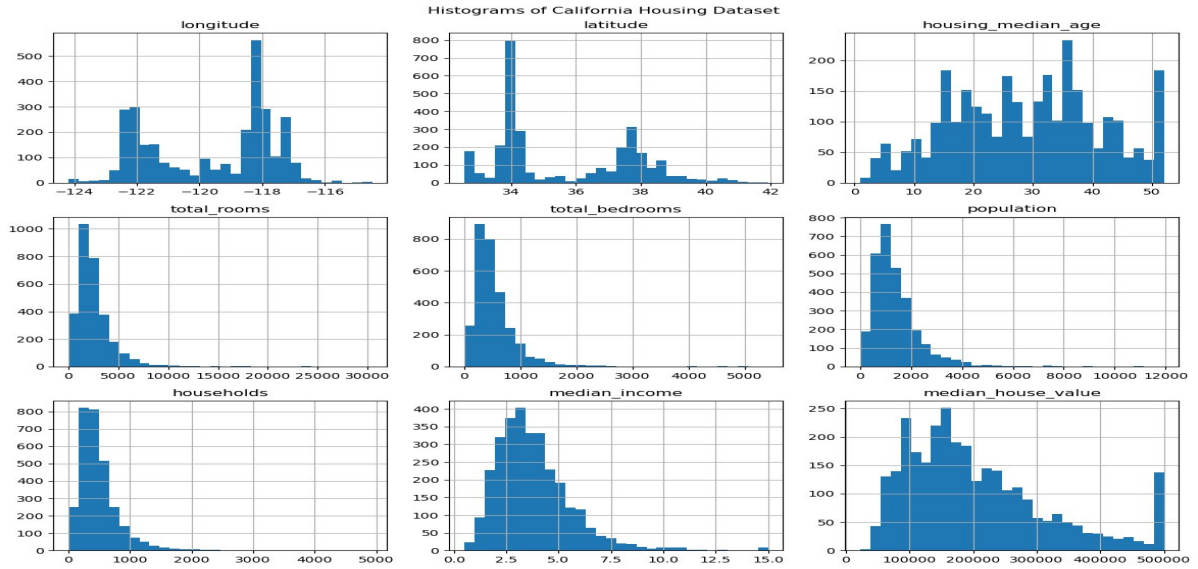
**OUTPUT-**

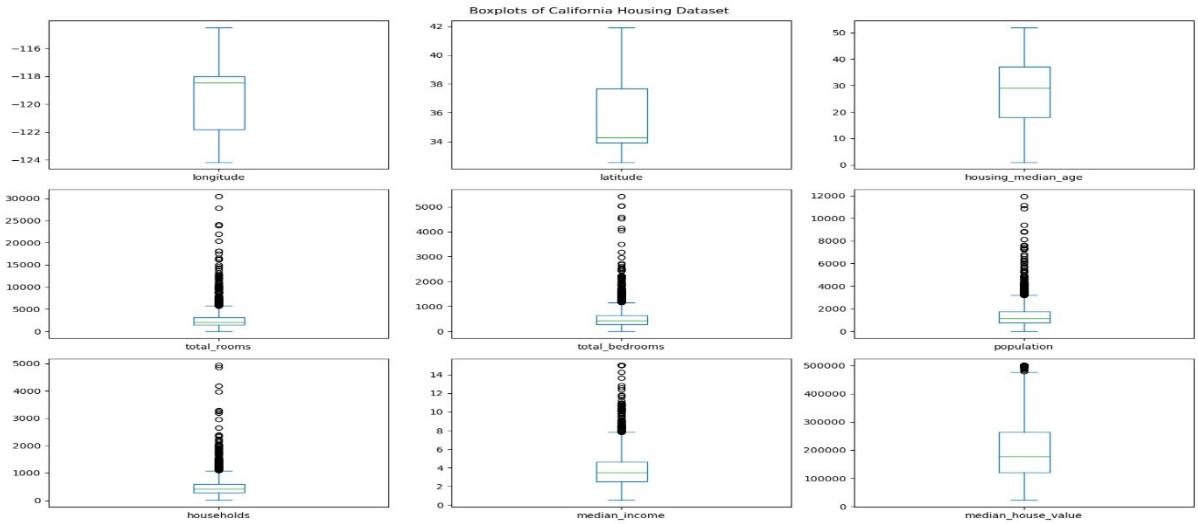
```
Dataset loaded successfully!
Shape of dataset: (3000, 9)

First 5 rows of dataset:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
```

| | | | | | |
|---|---|---|---|---|---|
| 0 | -122.05 | 37.37 | 27.0 | 3885.0 | 661.0 |
| 1 | -118.30 | 34.26 | 43.0 | 1510.0 | 310.0 |
| 2 | -117.81 | 33.78 | 27.0 | 3589.0 | 507.0 |
| 3 | -118.36 | 33.82 | 28.0 | 67.0 | 15.0 |
| 4 | -119.67 | 36.33 | 19.0 | 1241.0 | 244.0 |

| | population | households | median_income | median_house_value |
|---|---|---|---|---|
| 0 | 1537.0 | 606.0 | 6.6085 | 344700.0 |
| 1 | 809.0 | 277.0 | 3.5990 | 176500.0 |
| 2 | 1484.0 | 495.0 | 5.7934 | 270500.0 |
| 3 | 49.0 | 11.0 | 6.1359 | 330000.0 |
| 4 | 850.0 | 237.0 | 2.9375 | 81700.0 |



Training data size: (2400, 1)

Testing data size: (600, 1)

========== LINEAR REGRESSION ==========

MSE: 6355701945.13497

MAE: 60123.47952074411

R2 Score: 0.4953198289574967

========== POLYNOMIAL REGRESSION ==========

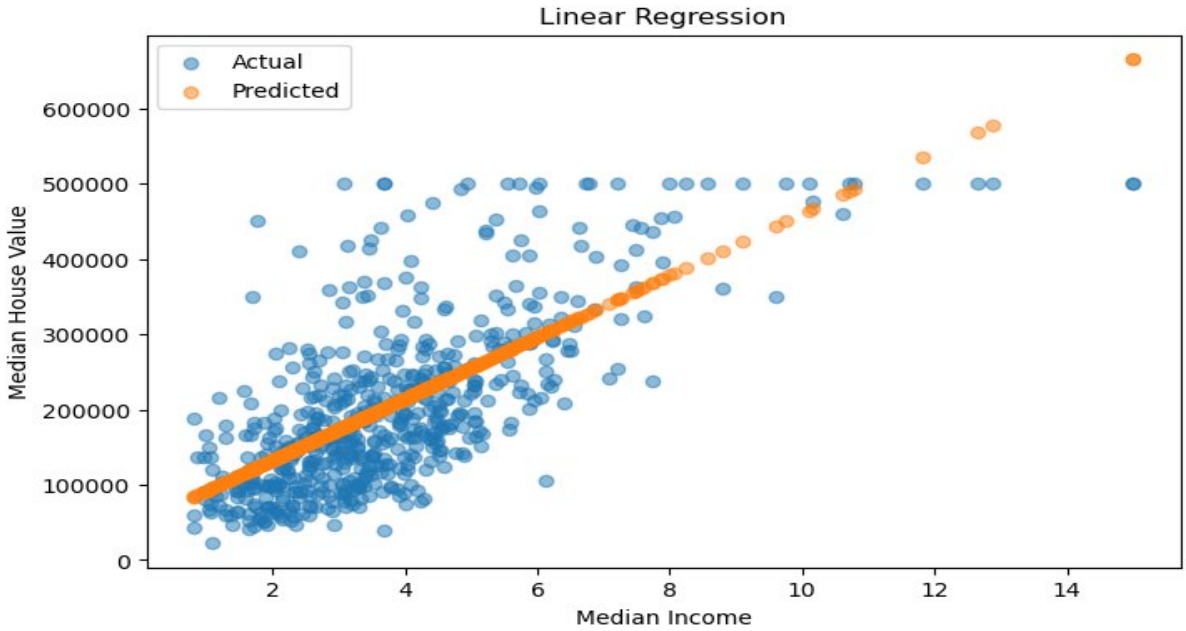
MSE: 6305388958.201255

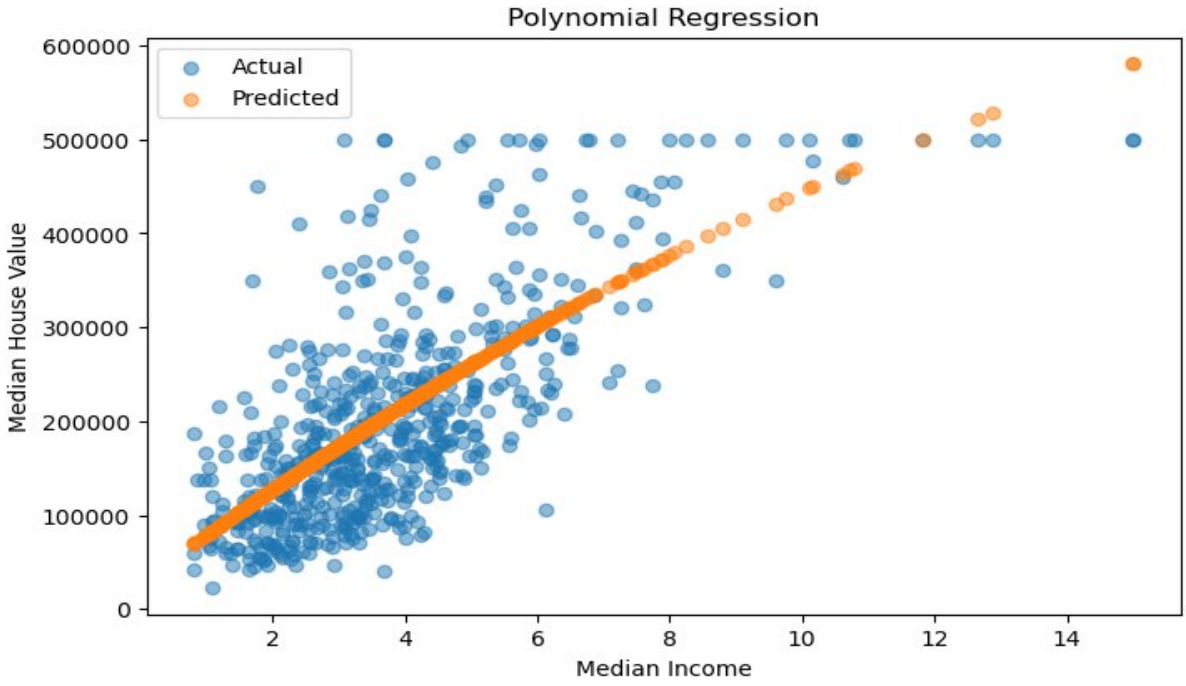
MAE: 59784.98749247233

R2 Score: 0.4993149764755145

========== MODEL COMPARISON ==========

| | Model | MSE | MAE | R2 Score |
|---|---|---|---|---|
| 0 | Linear Regression | 6.355702e+09 | 60123.479521 | 0.495320 |
| 1 | Polynomial Regression | 6.305389e+09 | 59784.987492 | 0.499315 |






Polynomial Regression performs better based on R2 Score.